INNOMATİCS
RESEARCH LABS

Deliverable 3: Technical Documentation

Name : Emmadi Srikar
Intern Id : IN226090702
Mail: srikaremmadi5@gmail.com

RAG-Based Customer Support Assistant With LangGraph & HITL

---

1. Introduction

What is RAG (Retrieval-Augmented Generation)?

RAG is a technique that combines information retrieval with language models. Instead of relying only on the model's training data, it retrieves relevant information from a knowledge base and uses it to generate accurate and contextaware responses. **Why RAG is needed**

Traditional chatbots:

- Provide generic or incorrect answers
- Cannot access updated or domain-specific knowledge

RAG solves this by:

- Retrieving relevant content dynamically
- Improving accuracy and reliability

Use Case Overview

This system acts as a customer support assistant that:

- Answers user queries based on a PDF knowledge base
- Uses semantic search for relevant information
- Escalates uncertain queries to a human (HITL)

---

2. System Architecture Explanation

The system follows a modular RAG architecture:

1. User provides a query via CLI
2. The query is passed to the LangGraph workflow
3. Retrieval layer fetches relevant document chunks from ChromaDB
4. LLM generates an answer using retrieved context

5. Decision node evaluates confidence
6. System either:
    - Returns answer
    - Escalates to human

Each component works independently, improving maintainability and scalability.

---

3. Design Decisions

1. Chunk Size Strategy

- Smaller chunks improve retrieval accuracy
- Larger chunks improve context
- A balanced chunk size is used to optimize both

2. Embedding Strategy

- HuggingFace embeddings are used
- Chosen for:
    - Open-source availability
    - Good semantic understanding
    - Easy integration

3. Retrieval Approach

- Top-K similarity search
- Relevance score filtering applied
- Ensures only meaningful chunks are used

4. Confidence Calculation

- Based on similarity score of retrieved documents
- Used to determine answer reliability

5. Prompt Design

- Combines:

    - User query
    - Retrieved context
- Ensures LLM generates grounded answers

---

4. Workflow Explanation (LangGraph)

LangGraph is used to control execution flow.

Nodes:

1. Processing Node

- Retrieves documents
- Generates answer using LLM

2. Decision Node

- Evaluates confidence score
- Determines whether to escalate

3. Output Node

- Returns final answer

4. Escalation Node

- Handles human fallback

State Transitions

The system maintains a shared state:

```
state = {
    "query": str,
    "docs": list,
    "answer": str,
    "confidence": float,
    "escalation_flag": bool
}
```

Each node updates the state accordingly.

---

5. Conditional Logic

Routing is based on:

1. Confidence Threshold

If confidence is below threshold → escalate

2. Response Validation

If answer contains “I don’t know” → escalate

Logic:

```
is_low_confidence = confidence < CONFIDENCE_THRESHOLD
is_unknown = "i don't know" in answer.lower()

if is_low_confidence or is_unknown:
    escalate = True
```

---

6. HITL Implementation

Role of Human Intervention

- Acts as fallback when system is uncertain
- Ensures user receives reliable response

When HITL is triggered

- Low confidence score
- No relevant documents
- Insufficient answer

Implementation

- Simulated using predefined response
- No real-time human integration

Benefits

- Improves reliability
- Prevents incorrect answers

Limitations

- Not connected to real human agents
- Response is static

---

7. Challenges & Trade-offs

1. Retrieval Accuracy vs Speed

- More chunks → better accuracy but slower
- Fewer chunks → faster but less accurate

2. Chunk Size vs Context Quality

- Small chunks → precise retrieval
- Large chunks → better context

3. Cost vs Performance

- High-quality LLM → better answers but higher cost
- Lightweight models → faster but less accurate

---

8. Testing Strategy

Testing Approach

- Manual testing using different query types

Sample Queries

1. Relevant query
   - "How do I return a product?"
     → Correct answer generated
2. Paraphrased query
   - "How can I send back a product?"
     → Correct answer

3. Irrelevant query
    - "What is AI?"
      → Escalation triggered

---

9. Future Enhancements

- Multi-document support
- Web-based UI (Streamlit)
- Chat history / memory
- Real-time human integration
- Deployment using cloud services

---

10. Summary

This system demonstrates a practical implementation of a RAG-based customer support assistant using LangGraph. It integrates retrieval, generation, and decision-making into a structured workflow, ensuring both accuracy and reliability through conditional routing and HITL escalation.